

## **TUTORIAL – 7: Boundary Value Analysis (BVA) & Equivalence Partitioning (EP)**

### **Aim**

To apply **Boundary Value Analysis (BVA)** and **Equivalence Partitioning (EP)** techniques for validation testing of forms and authentication modules.

### **Description**

**Boundary Value Analysis (BVA)** and **Equivalence Partitioning (EP)** are **Black Box Testing techniques** used to design effective test cases with fewer inputs while achieving maximum test coverage.

#### **1. Equivalence Partitioning (EP)**

Equivalence Partitioning divides the input data into **Valid** and **Invalid** partitions (groups). Only one value from each partition is tested because all values in the same partition are expected to behave similarly.

**Example: Username Length = 5 to 15 characters**

- **Valid Partition:** 5–15 characters
- **Invalid Partition:** Less than 5 characters
- **Invalid Partition:** More than 15 characters

Instead of testing every possible value (1–20), test only representative values such as **3, 10, and 16**.

#### **2. Boundary Value Analysis (BVA)**

Boundary Value Analysis focuses on the **boundary (edge) values**, where defects are most likely to occur.

For an input range of **5–15**, the important boundary values are:

- Minimum – 1 = **4**
- Minimum = **5**
- Minimum + 1 = **6**
- Maximum – 1 = **14**
- Maximum = **15**
- Maximum + 1 = **16**

### Difference Between EP and BVA

| Equivalence Partitioning                       | Boundary Value Analysis               |
|------------------------------------------------|---------------------------------------|
| Divides input into valid and invalid groups    | Tests boundary values                 |
| Reduces the number of test cases               | Detects boundary-related defects      |
| Tests one representative value from each group | Tests minimum and maximum edge values |
| Black Box Testing Technique                    | Black Box Testing Technique           |

### Problem Statement

Perform validation testing on the **Login Module** of the **Student Management System**, where the username length must be **5 to 15 characters**. Apply **Equivalence Partitioning** and **Boundary Value Analysis** to design test cases, execute them, identify defects (if any), and prepare a Bug Report.

### How to Perform This Practical (Student Procedure)

#### Step 1: Study the Requirement

Requirement:

- Username length must be **5 to 15 characters**.
- Username cannot be blank.
- Password cannot be blank.

#### Step 2: Identify Valid and Invalid Partitions

Input Range:

**Username Length = 5–15**

| Partition       | Type    |
|-----------------|---------|
| Less than 5     | Invalid |
| 5–15            | Valid   |
| Greater than 15 | Invalid |

### Step 3: Select Representative Values (EP)

Choose one value from each partition.

Example:

- 3 → Invalid
- 10 → Valid
- 16 → Invalid

### Step 4: Apply Boundary Value Analysis

Test the following values:

| Boundary    | Value |
|-------------|-------|
| Minimum – 1 | 4     |
| Minimum     | 5     |
| Minimum + 1 | 6     |
| Maximum – 1 | 14    |
| Maximum     | 15    |
| Maximum + 1 | 16    |

### Step 5: Execute Test Cases

Enter each username length in the application.

Compare:

- Expected Result
- Actual Result

Record:

- Pass
- Fail

### Step 6: Report Defects

If any invalid username is accepted or any valid username is rejected, prepare a Bug Report.

### Implementation Details

### Parameter Details

| Parameter               | Details                      |
|-------------------------|------------------------------|
| Project Name            | Student Management System    |
| Application/Software    | Student Management System    |
| Module Name             | Login Module                 |
| Version                 | Version 1.0                  |
| Programming Language    | Python                       |
| IDE/Tool Used           | Visual Studio Code / PyCharm |
| Database Used           | MySQL                        |
| Operating System        | Windows 10                   |
| Browser (If Applicable) | Google Chrome                |
| Test Environment        | Testing Environment          |
| Tester Name             | Student Name                 |
| Test Date               | DD/MM/YYYY                   |

### Source Code

```
username = input("Enter Username: ")
```

```
if 5 <= len(username) <= 15:
```

```
    print("Valid Username")
```

```
else:
```

```
    print("Invalid Username Length")
```

**Equivalence Class Table**

| <b>Input Field</b> | <b>Input Range / Condition</b> | <b>Equivalence Class Type</b> | <b>Test Value</b>             | <b>Expected Result</b> |
|--------------------|--------------------------------|-------------------------------|-------------------------------|------------------------|
| Username           | Less than 5 characters         | Invalid                       | abc (3 chars)                 | Invalid Username       |
| Username           | 5–15 characters                | Valid                         | student1 (8 chars)            | Valid Username         |
| Username           | More than 15 characters        | Invalid                       | studentmanagement1 (18 chars) | Invalid Username       |

**Boundary Value Analysis (BVA) Table**

| <b>Input Field</b> | <b>Minimum Value</b> | <b>Just Above Minimum</b> | <b>Nominal Value</b> | <b>Just Below Maximum</b> | <b>Maximum Value</b> | <b>Just Above Maximum</b> |
|--------------------|----------------------|---------------------------|----------------------|---------------------------|----------------------|---------------------------|
| Username Length    | 5                    | 6                         | 10                   | 14                        | 15                   | 16                        |

**Alternative BVA Format (Industry Standard)**

| <b>Input Field</b> | <b>Boundary Condition</b> | <b>Boundary Value</b> | <b>Expected Result</b> | <b>Actual Result</b> | <b>Status</b> |
|--------------------|---------------------------|-----------------------|------------------------|----------------------|---------------|
| Username           | Minimum – 1               | 4                     | Invalid Username       | Invalid Username     | Pass          |
| Username           | Minimum                   | 5                     | Valid Username         | Valid Username       | Pass          |
| Username           | Minimum + 1               | 6                     | Valid Username         | Valid Username       | Pass          |

|          |             |    |                  |                  |      |
|----------|-------------|----|------------------|------------------|------|
| Username | Maximum – 1 | 14 | Valid Username   | Valid Username   | Pass |
| Username | Maximum     | 15 | Valid Username   | Valid Username   | Pass |
| Username | Maximum + 1 | 16 | Invalid Username | Invalid Username | Pass |

### Bug Identification Details

| Field                     | Details                                                                     |
|---------------------------|-----------------------------------------------------------------------------|
| <b>Bug ID</b>             | BUG-007                                                                     |
| <b>Defect Title</b>       | Username length validation issue                                            |
| <b>Module Name</b>        | Login Module                                                                |
| <b>Feature Affected</b>   | Username Validation                                                         |
| <b>Defect Description</b> | Application accepts username with more than 15 characters (example defect). |
| <b>Defect Type</b>        | Validation Defect                                                           |
| <b>Severity</b>           | Medium                                                                      |
| <b>Priority</b>           | High                                                                        |
| <b>Environment</b>        | Windows 10 / Chrome                                                         |
| <b>Reported By</b>        | Student Name                                                                |
| <b>Date Reported</b>      | DD/MM/YYYY                                                                  |

**Note:** If your program correctly rejects usernames longer than 15 characters, record "**No Defect Found**" instead of creating a bug report.

### Steps to Reproduce

| Step No. | Action Performed | Input/Test Data               | Expected Result              | Actual Result                                |
|----------|------------------|-------------------------------|------------------------------|----------------------------------------------|
| 1        | Open Login Page  | -                             | Login page displayed         | Login page displayed                         |
| 2        | Enter Username   | studentmanagement1 (18 chars) | Username should be rejected  | Username accepted<br><i>(example defect)</i> |
| 3        | Click Login      | Click                         | Validation message displayed | Username accepted                            |

### Bug Report Template

| Field              | Details                                                  |
|--------------------|----------------------------------------------------------|
| Bug ID             | BUG-007                                                  |
| Bug Summary        | Username accepts more than 15 characters                 |
| Module Name        | Login Module                                             |
| Severity           | Medium                                                   |
| Priority           | High                                                     |
| Steps to Reproduce | Enter username longer than 15 characters and click Login |
| Expected Result    | Username should be rejected                              |
| Actual Result      | Username accepted <i>(example defect)</i>                |
| Assigned To        | Developer                                                |
| Status             | Open                                                     |

|                      |                                    |
|----------------------|------------------------------------|
| <b>Resolution</b>    | Pending                            |
| <b>Retest Status</b> | Not Yet Tested                     |
| <b>Remarks</b>       | Length validation needs correction |

### Bug Summary

| Bug ID  | Module | Severity | Priority | Status |
|---------|--------|----------|----------|--------|
| BUG-007 | Login  | Medium   | High     | Open   |

### Observation

| Sr. No. | Observation                                            | Remarks                                     |
|---------|--------------------------------------------------------|---------------------------------------------|
| 1       | Valid username accepted                                | Pass                                        |
| 2       | Username shorter than 5 rejected                       | Pass                                        |
| 3       | Username longer than 15 rejected                       | Pass <i>(or Defect Found if applicable)</i> |
| 4       | Boundary values tested successfully                    | Completed                                   |
| 5       | EP and BVA techniques reduced the number of test cases | Completed                                   |

### Output

Boundary Value Analysis Completed

Input Tested

Length = 4 → Invalid Username

Length = 5 → Valid Username

Length = 6 → Valid Username

Length = 14 → Valid Username



Length = 15 → Valid Username  
Length = 16 → Invalid Username

Equivalence Partitioning Completed

Valid Partition Tested  
Invalid Partitions Tested

Overall Result:  
All Test Cases Passed Successfully

**Solve the Below Mention Questionnaires**

**1. Equivalence Partitioning is a type of:**

- A) White Box Testing
- B) Black Box Testing**
- C) Performance Testing
- D) Security Testing

**Answer: B) Black Box Testing**

**2. What is the main objective of Equivalence Partitioning?**

- A) Increase number of test cases
- B) Reduce test cases while maintaining coverage**
- C) Improve coding standards
- D) Increase execution time

**Answer: B) Reduce test cases while maintaining coverage**

**3. Boundary Value Analysis focuses on:**

- A) Random Values
- B) Internal Code Structure
- C) Boundary Values of Input Data**
- D) Database Design

**Answer: C) Boundary Values of Input Data**

**4. For a username length range of 5–15, which value is just below the minimum boundary?**

- A) 3
- B) 4**
- C) 5
- D) 6

**Answer: B) 4**

**5. Which testing technique divides input data into valid and invalid groups?**

- A) Regression Testing
- B) Integration Testing
- C) Equivalence Partitioning**
- D) Acceptance Testing

**Answer: C) Equivalence Partitioning**

#### **Conclusion / Outcome**

- Understood the concepts of **Equivalence Partitioning** and **Boundary Value Analysis** as Black Box Testing techniques.
- Designed optimized test cases using valid and invalid equivalence classes.
- Applied Boundary Value Analysis to verify minimum and maximum input limits.
- Executed validation test cases and compared expected and actual results.
- Learned how EP and BVA reduce the number of test cases while improving test coverage and helping identify input validation defects.